

TemSeg Overview Draft 1

Abstract—Quantitative characterization of nanoparticle size and shape distributions from Transmission Electron Microscopy (TEM) images remains a critical bottleneck in materials science workflows. Existing approaches rely on manual annotation on legacy software that trades accuracy for throughput. We present TEMseg, an open-source desktop application that combines deep learning-based instance segmentation with interactive refinement and automated analysis.

The application supports multiple domain finetuned models varied across sizes for flexibility and provides a human-in-the-loop workflow where researchers can refine segmentation results through direct contour manipulation, single-click particle splitting, and region-based re-segmentation. Per-particle morphological statistics are computed from exact pixel masks with automatic physical unit calibration from microscope metadata. TEMseg ships as a self-contained native application requiring no programming expertise, reducing analysis time from hours of manual work to minutes of interactive review.

I. FEATURES

A. Image Upload

The application supports two common image upload mechanisms: users may either click the image workspace to open the system file picker or drag and drop files directly into the upload area. Currently supported formats include TIF/TIFF, EMD, JPG/JPEG, PNG, and NumPy array files. Where available, metadata embedded within the image file such as pixel size and scale information is automatically extracted and stored for downstream analysis. Clicking the application logo clears the current workspace, removes the loaded image, and resets the workflow state.

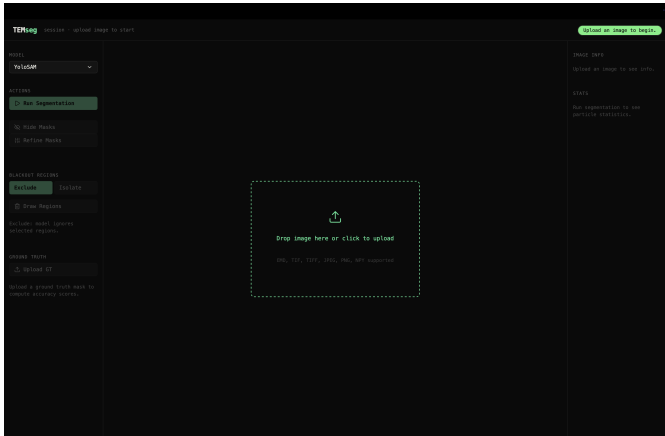


Fig. 1. Clicking open desktop native file explorer; click a valid file or drag and drop.

B. Choosing models

The initial goal of this tool was to provide accessible support for state-of-the-art particle segmentation models, especially for users without dedicated GPU hardware. At

present, the application provides access to two segmentation pipelines as can be seen in Fig. 2:

a) YoloSAM by Ardra Genc et al.:

YoloSAM is a dual-model segmentation pipeline that combines YOLOv8 for object detection with Segment Anything Model (SAM) for dense segmentation. YOLOv8 first detects regions of interest by generating bounding boxes, which are then passed to SAM to generate instance-segmented particle masks. The models were fine-tuned by Genc et al. on a custom annotated TEM dataset, described in detail in their work. We use their published YOLOv8 weights together with the upstream SAM ViT-B checkpoint without modification. Our contribution focuses on improving runtime efficiency of the original pipeline while preserving output quality and segmentation accuracy. Three main optimizations were introduced:

1) ONNX Runtime acceleration for YOLO inference

The original PyTorch YOLOv8 model was exported once to ONNX format during packaging. Inference is then executed through ONNX Runtime, which performs graph-level optimizations such as operator fusion and constant folding while using CPU-optimized kernels. On our 31-image CPU benchmark, this reduced YOLO inference time from 1.52 ± 0.15 s to 1.09 ± 0.15 s (approximately $1.4 \times$ faster) without affecting detection outputs.

2) Persistent SAM predictor reuse

In the reference implementation, the SAM checkpoint and predictor are reloaded for every segmentation call. In our implementation, the predictor is initialized once at process startup and reused across images and segmentation runs. This removes repeated checkpoint loading and device transfer overhead, reducing end-to-end runtime from 7.11 ± 3.84 s to 6.05 ± 2.59 s per image.

3) Caching of SAM image embeddings

The SAM image encoder, based on a ViT-B backbone, dominates runtime cost. During interactive workflows, users frequently re-run segmentation on the same image while adjusting parameters. Instead of recomputing image embeddings each time, our pipeline caches the encoded image features and reuses them for subsequent runs. Only the lightweight decoder stage is recomputed. This reduces repeat-call SAM runtime from 4.48 ± 2.28 s to 1.43 ± 1.70 s ($3.1 \times$ faster), and overall re-run time from 5.98 ± 2.32 s to 2.47 ± 1.70 s ($2.4 \times$ faster).

All implementation related improvement are summarized in the table below:

Category	Original model by Genc et al.	TemSeg implementation
YOLOv8 inference backend	PyTorch eager mode on the published best12x.pt checkpoint.	best12x.onnx exported once at packaging, served via ONNX Runtime with ahead-

Category	Original model by Genc et al.	TemSeg implementation
		of-time graph optimisation and CPU-tuned kernels. <i>Measured: 1.52 ± 0.15 s $\rightarrow 1.09 \pm 0.15$ s per image ($1.4\times$ faster, $n = 31$).</i>
SAM predictor lifecycle	sam_model_registry(...) is rebuilt, moved to device, and wrapped in a fresh SamPredictor on every segmentation call.	[Fixed] Predictor is constructed once at process start and reused for all subsequent calls; weight load and host-to-device transfer cost is paid once. <i>Measured: removes ≈ 1 s of per-image overhead (7.11 s $\rightarrow 6.05$ s end-to-end).</i>
SAM image-embedding cache	No cache. The ViT-B image encoder runs from scratch on every call, even when the user re-segments the same image during refinement.	Encoder output (features, original_size, input_size) is returned with the segmentation result and threaded back into the next call to bypass the encoder entirely; only the prompt-conditioned mask decoder runs on reruns. <i>Measured: SAM stage 4.48 s $\rightarrow 1.43$ s on rerun ($3.1\times$); end-to-end rerun 5.98 s $\rightarrow 2.47$ s ($2.4\times$).</i>
Memory robustness on dense images	Single predict_torch call over all detections, which allocates an $(N, 1, H, W)$ mask tensor; exhausts RAM on images with thousands of particles.	Detections are processed in chunks of 64 boxes with incremental max-reduction over partial masks; bounded peak memory regardless of particle count.
Patch-wise inference for large images	Not provided; whole-image inference only.	segment_batch() accepts a list of image patches with offsets, runs

Category	Original model by Genc et al.	TemSeg implementation
		YOLO and SAM per patch, and stitches masks back into the full image.
Output equivalence	—	Verified per image: median mask IoU vs. the reference implementation II. 0.98; MEAN IoU AGAINST HAND-LABELLED GROUND TRUTH 0.841 ± 0.062 (OURS) vs. 0.840 ± 0.064 (reference) — within statistical noise of identical.

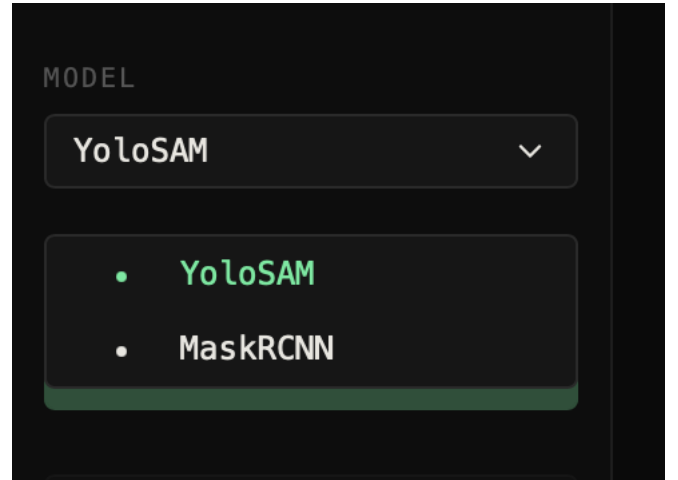


Fig. 2. Drop-down menu to easily switch between models used for segmentation.